

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Object-Oriented Programming in JavaScript: Functions vs. Classes and Object Instantiation

2. AIM

To differentiate between JavaScript functions and classes, and to design and develop a JavaScript program utilizing ES6 classes to instantiate multiple student objects sharing common properties and methods.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the fundamental difference between procedural functions, constructor functions, and ES6 class syntax in JavaScript.
 - To define a blueprint class with a constructor method to initialize instance properties dynamically.
 - To attach shared prototype methods to a class to achieve memory efficiency across multiple objects.
 - To instantiate multiple distinct object instances (e.g., student1, student2, student3) using the 'new' keyword.
 - To invoke class methods on individual instances to verify property encapsulation and method execution.
-

4. THEORY

Difference Between JavaScript Functions and Classes

In JavaScript, functions and classes are both essential building blocks, but they serve distinct architectural paradigms:

Feature / Aspect	JavaScript Functions	JavaScript Classes
Definition & Primary Role	A block of reusable code designed to perform a specific computational task or return a value.	A blueprint/template introduced in ES6 for creating objects with shared properties and methods (OOP).
Invocation / Instantiation	Invoked directly via <code>functionCall()</code> or can act as constructor when invoked with 'new'.	Must be instantiated using the 'new' keyword; calling a class without 'new' throws a <code>TypeError</code> .
Hoisting Behavior	Function declarations are hoisted completely to the top of their scope, allowing invocation before definition.	Classes are NOT hoisted like functions. Accessing a class before its declaration results in a <code>ReferenceError</code> .
Constructor Method	Uses the function body itself as the constructor to assign properties via 'this'.	Uses a dedicated <code>constructor()</code> method explicitly defined within the class body.
Method Storage & Prototypes	Methods assigned via 'this.method' are duplicated in every object, consuming more memory.	Methods defined in a class body automatically attach to the class prototype, sharing one copy across all instances.
Inheritance Syntax	Implemented manually via prototype chaining (<code>Object.create</code> , <code>Function.prototype</code>).	Provides clean syntactic sugar using the 'extends' and 'super' keywords.

Creating Multiple Objects Using Classes

A class acts as an architectural blueprint. When a class is defined, it encapsulates state (properties) and behavior (methods):

1. **Constructor Initialization:** The `constructor(name, rollNo, branch)` method is executed automatically whenever `'new Student(...)'` is called. It binds unique property values to the specific object instance via the 'this' keyword.
2. **Shared Prototype Methods:** Methods such as `displayDetails()` are declared inside the class body. Instead of duplicating the function in memory for every instance, JavaScript attaches it to `Student.prototype`. All instances share access to this single method.
3. **Multiple Instantiations:** Using `'const student1 = new Student(...)'`, `'const student2 = new Student(...)'`, and `'const student3 = new Student(...)'`, multiple independent object instances are created, each maintaining its own unique state while sharing common methods.

5. ALGORITHM / PROCEDURE

- Create a JavaScript source file named `studentClass.js`.
- Define a class named `Student` using the ES6 class syntax.
- Inside the class, create a constructor accepting three parameters: `name`, `rollNo`, and `branch`.

- Bind the incoming parameters to instance properties using `this.name = name`, `this.rollNo = rollNo`, and `this.branch = branch`.
 - Define a class method `displayDetails()` that logs the formatted properties to the console.
 - Instantiate `student1` with values `("Rahul Sharma", "A24126552001", "CSE")` using the new keyword.
 - Instantiate `student2` with values `("Priya Patel", "A24126552002", "IT")`.
 - Instantiate `student3` with values `("M. Purna Chandra", "A24126552095", "CSE")`.
 - Invoke the `displayDetails()` method on `student1`, `student2`, and `student3` sequentially.
 - Execute the script using Node.js (`node studentClass.js`) and record output.
-

6. PROGRAM

studentClass.js

// Example of a JavaScript Function

```
function add(a, b) {  
    // Function performs a specific task  
    return a + b;  
}
```

```
console.log("Sum:", add(10, 20));
```

// Example of a JavaScript Class

```
class Student {  
  
    // Constructor initializes object properties  
    constructor(name, rollNo, branch) {  
        this.name = name;  
        this.rollNo = rollNo;  
        this.branch = branch;  
    }  
  
    // Method common to all Student objects  
    displayDetails() {  
        console.log("Name:", this.name);  
        console.log("Roll No:", this.rollNo);  
        console.log("Branch:", this.branch);  
    }  
}
```

// Creating multiple objects from the same class

```
let student1 = new Student("Rahul", 101, "CSE");
let student2 = new Student("Priya", 102, "AI & ML");
let student3 = new Student("Arun", 103, "ECE");
```

```
// Calling the common method for each object
```

```
console.log("Student 1:");
student1.displayDetails();
```

```
console.log("\nStudent 2:");
student2.displayDetails();
```

```
console.log("\nStudent 3:");
student3.displayDetails();
```

7. CODE EXPLANATION

Code / Component	Explanation
class Student { ... }	Declares an ES6 class blueprint for student entity representation.
constructor(name, rollNo, branch)	Special constructor method invoked automatically upon object creation.
this.name = name	Binds the passed parameter to the specific object instance being created.
displayDetails() { ... }	Defines an instance method stored on Student.prototype shared across all instances.
new Student(...)	Instantiates a new Student object, allocating memory and triggering the constructor.
student1.displayDetails()	Executes the prototype method in the context of the student1 object instance.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Call add(10, 20)	Returns 30	Sum printed as 30	Pass
TC-02	Call add(5, 15)	Returns 20	Sum calculated correctly	Pass
TC-03	Create student1 object and call student1.displayDetails()	Displays Name: Rahul, Roll No: 101, Branch: CSE	Details printed correctly	Pass
TC-04	Create student2 object and call student2.displayDetails()	Displays Name: Priya, Roll No: 102, Branch: AI & ML	Details printed correctly	Pass
TC-05	Create student3 object and call student3.displayDetails()	Displays Name: Arun, Roll No: 103, Branch: ECE	Details printed correctly	Pass
TC-06	Execute complete program	Displays sum and details of all three students in sequence	Output displayed as expected	

9. OUTPUT

Output 1: Call add(10, 20)

```
PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-5> node functions-classes.js  
Sum: 30
```

Output 2: Call add(5, 15)

```
Sum: 20
```

Output 3: call student1.displayDetails()

```
Student 1:  
Name: Rahul  
Roll No: 101  
Branch: CSE
```

Output 4: call student2.displayDetails()

```
Student 2:  
Name: Priya  
Roll No: 102  
Branch: AI & ML
```

Output 5: call student3.displayDetails()

```
Student 3:  
Name: Arun  
Roll No: 103  
Branch: ECE
```

Output 6: Complete Execution

```
Branch: ECE
● PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-5> node functions-classes.js
Sum: 30

Sum: 20

Student 1:
Name: Rahul
Roll No: 101
Branch: CSE

Student 2:
Name: Priya
Roll No: 102
Branch: AI & ML

Student 3:
Name: Arun
Roll No: 103
Branch: ECE
○ PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-5> █
```

10. RESULT

Thus, the concepts of JavaScript functions and classes were comprehensively differentiated, and an ES6 Student class was successfully designed and developed. Multiple object instances were instantiated with unique state properties while sharing prototype methods. All test cases passed with expected outputs, demonstrating core object-oriented principles in JavaScript.